

records in java :

record is a keyword in java which was introduced from JDK 17V.

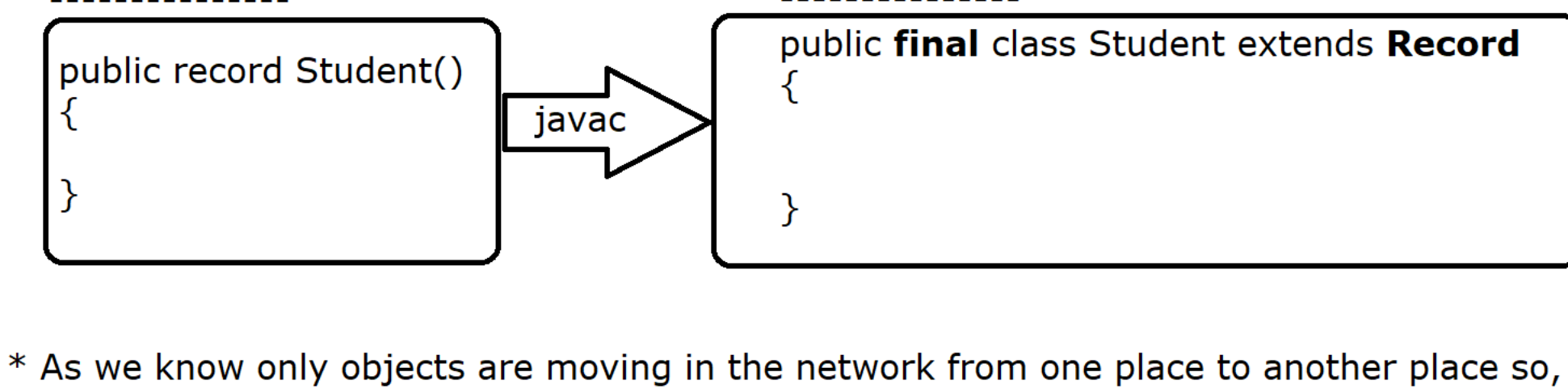
* We have a predefined class called **Record** available in java.lang package. It is an abstract class.

Example :

```
public abstract class Record extends Object
{
}
```

* record is mainly used to overcome from boiler-plate code while writing the BLC OR POJO (Plain Old Java Object) OR Data Carrier (Carry the data from one place to another place) class.

* A record is similar to a class because java compiler will **automatically generate a class** in the .class file.



* As we know only objects are moving in the network from one place to another place so, If we want to write a BLC class as a **immutable Data carrier class then we should use the following :**

```
Employee e1 = new Employee(101, "Scott");
Validator.validate(e1);
```

Data carrier OR POJO OR BLC OR DTO (Data transfer object)

```
public class Employee
{
    //Non static fields
    //Parameterized Constructor
    //toString()
    //setter & getter
    //hashCode() & equals(Object obj)
}

public class Validator
{
    public static void validate(Employee emp)
    {
    }
}
```

Records are immutable data carrier so, now with the help of record we can send our immutable data (final data) from one application to another application.

It is also known as DTO (Data transfer object) OR POJO (Plain Old Java Object) classes.

It is mainly used to concise our code as well as remove the boiler plate code.

In record, we need to declare the constructor at the time of declaration of record which is known as canonical constructor and the variables which are known as components are by default private and final.

In order to validate the outer world data, we can write our own constructor which is known as compact constructor.

Record will automatically generate the implementation of toString(), equals(Object obj) and hashCode() method. Compiler will automatically add getter method which depends upon the name of the componenets. (int id -> id())

We can define static and non static method as well as static variable and static block inside the record. We cannot define instance variable and instance block inside the record.

We can't extend or inherit records because by default every record is implicitly final and It is extending from java.lang.Reocrd class, which is an abstract class.[MI is not supported]

We can implement an interface by using record.

We don't have setter facility in record because by default components are final as well as we cannot initialize the components explicitly inside the record class.

```
//Program:
-----
package com.ravi.record_demo;

public class EmployeeClass
{
    private int id;
    private String name;

    public EmployeeClass(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString()
    {
        return "EmployeeClass [id=" + id + ", name=" + name + "]";
    }

    public int getId()
    {
        return id;
    }

    public void setId(int id)
    {
        this.id = id;
    }

    public String getName()
    {
        return name;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    @Override
    public int hashCode()
    {
        return this.id+this.name.hashCode();
    }

    @Override
    public boolean equals(Object obj)
    {
        if(obj instanceof EmployeeClass)
        {
            EmployeeClass e2 = (EmployeeClass) obj;

            if(this.id==e2.id && this.name.equals(e2.name))
            {
                return true;
            }
            else
            {
                return false;
            }
        }
        else
        {
            return false;
        }
    }
}

package com.ravi.record_demo;

public record EmployeeRecord(int id, String name) //Canonical constructor (Components -> private final)
{
    //Compact constructor
    public EmployeeRecord
    {
        if(id<=0)
        {
            System.err.println("Id cannot be 0 OR negative");
            System.exit(0);
        }
    }
}

package com.ravi.record_demo;

public class ComparisonDemo
{
    public static void main(String[] args)
    {
        //class
        EmployeeClass c1 = new EmployeeClass(111, "Scott");
        IO.println(c1);
        IO.println("Id & Name is :"+c1.getId()+" , "+c1.getName());
        EmployeeClass c2 = new EmployeeClass(111, "Scott");
        IO.println(c1.equals(c2));
        IO.println(c1.hashCode()+" : "+c2.hashCode());

        IO.println(".....");
        //record
        EmployeeRecord r1 = new EmployeeRecord(999, "Raj");
        IO.println(r1);
        IO.println("Id & Name is :"+r1.id()+" , "+r1.name());
        EmployeeRecord r2 = new EmployeeRecord(999, "Raj");
        IO.println(r1.equals(r2));
        IO.println(r1.hashCode()+" : "+r2.hashCode());
    }
}
```